Project Overlord — Functional Specification

Application: Project Overlord: Business is Booming!
Source file: `overlord-003.html`
Spec version: 1.0
Purpose: This document is a behavioral specification of the Project Overlord web application. It serves as the source of truth for test case generation and analysis. It describes what the system does, what states it can be in, and what rules govern transitions between those states — without reference to implementation details.

---

1. System Overview

Project Overlord is a single-page browser application that simulates a configurable bomb defusal scenario. A user provisions a fictional device with custom codes and a countdown duration, then interacts with it by entering codes to activate and deactivate it. The device detonates — transitioning to a terminal failure state — if either the countdown expires while the device is active, or the user exhausts their allowed deactivation attempts with incorrect codes.

The application is entirely client-side. There is no server, no persistence, and no state that survives a page reload.

---

2. Application Screens

The application has three mutually exclusive screens. Only one screen is visible at any time.

Screen ID	Name	Entry Condition	Exit Condition
`setup-screen`	Setup / Provision	Application load (default)	Successful form submission
`bomb-screen`	Bomb	Successful provisioning	Detonation (timer expires or attempts exhausted)
`detonation-screen`	Detonation	Either detonation condition is met	Page reload only (no in-app navigation back)

Transitions between screens are **one-way**. The only way to return to the Setup screen is to reload the page, which resets all state entirely.

---

3. Setup Screen

3.1 Purpose

Allows the user to configure the bomb before interacting with it. All three fields are optional; each falls back to a default value if left empty.

3.2 Fields

Field ID	Label	Type	Default	Constraints
`activation-code`	Activation Code	text	`1234`	Must be exactly 4 numeric digits
`deactivation-code`	Deactivation Code	text	`0000`	Must be exactly 4 numeric digits
`countdown-duration`	Countdown Duration (seconds)	number	`30`	Must be a positive integer (minimum: 1)

3.3 Validation Behavior

- If the activation code field is empty on submission, the value `1234` is used.
- If the deactivation code field is empty on submission, the value `0000` is used.
- If the countdown duration field is empty on submission, the value `30` is used.
- If a non-empty activation code does not match the pattern `^\d{4}$`, the form is rejected with an alert: `"Activation code must be 4 digits"`.
- If a non-empty deactivation code does not match the pattern `^\d{4}$`, the form is rejected with an alert: `"Deactivation code must be 4 digits"`.
- If a countdown value is provided but is less than `1`, the form is rejected with an alert: `"Countdown must be at least 1 second"`.

3.4 Notable Behaviors

- The activation code and deactivation code **may be identical**. The application does not enforce that they differ.
- The `maxlength="4"` HTML attribute on the code fields restricts browser-level entry to 4 characters, but this is a UI constraint only — it does not replace validation logic.
- The countdown field accepts non-integer numeric input (e.g., `10.7`), but this is converted to an integer via `parseInt`, which truncates the decimal portion.

---

4. Core Entities

4.1 Trigger

The Trigger manages activation state and tracks incorrect deactivation attempts.

State model:

State	Description
Inactive	Default state. Bomb is not counting down.
Active	Bomb is counting down. Deactivation is possible.
Detonated	More than 2 incorrect deactivation attempts have been made. Terminal state.

Rules:

- A Trigger starts in the **Inactive** state.
- The Trigger becomes **Active** when the correct activation code is submitted while Inactive.
- An incorrect activation code has no state effect and does not increment any counter.
- Each deactivation attempt (whether correct or incorrect) **increments the attempt counter**.
- The Trigger becomes **Detonated** when $\boxed{\text{attempts} > 2}$ — meaning detonation occurs on the **3rd failed attempt** (attempts 1, 2, 3).
- A correct deactivation code resets the Trigger to Inactive and resets the attempt counter to 0.
- Deactivation cannot be attempted if the Trigger is already in the Detonated state.
- The attempt counter is **not reset** by incorrect deactivation attempts — only a successful deactivation resets it.

Attempt counter and detonation threshold:

Attempts Made	Attempts Remaining (displayed)	Detonated?
0	3	No
1	2	No
2	1	No
3	0	Yes

> **Important note:** The error message displayed to the user reads `"Incorrect deactivation code (N attempts remaining)"` where N = `3 - attempts`. This calculation runs *after* the attempt is recorded, so on the third failed attempt the message would read `0 attempts remaining` — and detonation occurs.

4.2 Timer

The Timer tracks elapsed countdown time.

State model:

State	Description
Stopped	Default. Time is not advancing.
Running	Time is advancing. Started when the bomb is activated.
Expired	`timeRemaining` has reached 0 while Running. Terminal state.

Rules:

- The Timer starts in the **Stopped** state with `timeRemaining` set to the configured countdown.
- The Timer begins **Running** when `start()` is called (triggered by successful activation).
- If `stop()` is called while Running (triggered by successful deactivation), the Timer returns to **Stopped** and captures the remaining time at that moment.
- If the Timer is already Stopped, calling `stop()` has no effect.
- If the Timer is already Running, calling `start()` has no effect.
- The Timer reaches **Expired** when `Date.now() >= detonationTime`. This is evaluated every 100ms by the UI polling loop.
- When 10 or fewer seconds remain **and the Timer is Running**, the timer display enters a blinking "warning" animation.

4.3 Bomb

The Bomb is a composite of one Trigger and one Timer. It has no state of its own beyond what its components hold.

Detonation condition: The Bomb is considered detonated if **either** of the following is true:

1. `trigger.isDetonated()` — the attempt counter exceeds 2
2. `timer.isDetonated()` — the current time has passed the detonation timestamp

Both conditions are evaluated continuously every 100ms while on the Bomb screen.

5. Bomb Screen

5.1 Layout

The Bomb screen has three visual panels:

- **Trigger panel** (left): Status indicator, error message area, code display, and keypad
- **Timer panel** (center): Countdown display
- **Device panel** (right): SVG bomb graphic with embedded time readout

5.2 Code Entry

The user may enter a 4-digit code by either:

- Clicking the numeric keypad buttons (digits 0–9)
- Typing directly into the code input field

Both methods maintain a shared internal `codeStack` (an array of digit characters). The code display reflects the current contents of `codeStack`.

Keypad constraints:

- The keypad only appends digits when `codeStack.length < 4`.
- The `⌫` (delete) button removes the last digit from `codeStack`.

Direct input constraints:

- Non-numeric characters are stripped on input.
- Input is capped at 4 characters.
- Typing in the field overrides the `codeStack` entirely (it does not append to it).

Submitting the code:

- Clicking the **Action button** (`Activate` or `Deactivate`)
- Pressing **Enter** while the code input field is focused

After any submission attempt (successful or not), `codeStack` is cleared and the code display is reset to empty.

5.3 Action Button

The Action button label reflects the current bomb state:

Bomb State	Button Label
Inactive	Activate
Active	Deactivate

5.4 Status Indicator

Bomb State	Indicator Text	Visual Style
Inactive	Bomb is Inactive	Green, pulsing
Active	Bomb is Active	Red, pulsing

5.5 Error Messages

Error messages appear in the error message area above the code display. They are cleared on each new submission attempt.

Condition	Message
Code submitted with fewer than 4 digits	"Code must be 4 digits"
Incorrect activation code entered	"Incorrect activation code"
Incorrect deactivation code entered	"Incorrect deactivation code (N attempts remaining)"

5.6 Timer Display

- The timer updates every 100ms.
- Format is MM:SS (zero-padded), e.g., 01:05 for 65 seconds.
- The SVG bomb graphic also displays the raw seconds remaining (e.g., 65).
- When 10 or fewer seconds remain and the timer is running, the timer display enters a blinking warning state.
- When the bomb is Inactive (timer stopped), the display shows the remaining time without blinking.

---

6. State Transition Table

This table captures all meaningful transitions in the application.

Current State	User Action / Event	Result State	Side Effects
Setup screen	Submit valid form	Bomb screen (Inactive)	Bomb provisioned with configured values
Setup screen	Submit form with invalid codes or duration	Setup screen	Alert shown; no bomb created
Bomb Inactive	Enter correct activation code + submit	Bomb Active	Timer starts; status turns red; button changes to "Deactivate"
Bomb Inactive	Enter incorrect code + submit	Bomb Inactive	Error shown; code cleared
Bomb Inactive	Enter fewer than 4 digits + submit	Bomb Inactive	Error: "Code must be 4 digits"; code cleared
Bomb Active	Enter correct deactivation code + submit	Bomb Inactive	Timer stops; status turns green; button changes to "Activate"
Bomb Active	Enter incorrect deactivation code (attempt 1)	Bomb Active	Error shown with "2 attempts remaining"; code cleared
Bomb Active	Enter incorrect deactivation code (attempt 2)	Bomb Active	Error shown with "1 attempt remaining"; code cleared
Bomb Active	Enter incorrect deactivation code (attempt 3)	Detonation screen	Bomb detonated; interval cleared; screen transitions
Bomb Active	Timer reaches 0	Detonation screen	Bomb detonated; interval cleared; screen transitions
Detonation screen	Click "Reset Everything"	Setup screen	Page reloads; all state lost

7. Detonation Conditions (Summary)

There are exactly two independent paths to detonation:

1. **Attempt exhaustion:** The user enters an incorrect deactivation code 3 times while the bomb is Active. This is tracked by the Trigger independently of the Timer.
2. **Timer expiry:** The bomb is Active and the countdown reaches zero. This is tracked by the Timer independently of the Trigger.

Both conditions are evaluated together every 100ms. Whichever occurs first causes the transition to the Detonation screen.

---

8. Detonation Screen

- Displays the message: `"That went as well as could be expected..."`
- Displays an explosion emoji animation
- Displays: `"BOOM! Explosion occurred."`
- Provides a single **Reset Everything** button that triggers `location.reload()`, returning the user to the Setup screen with all state cleared.

---

9. Edge Cases and Notable Behaviors

These are behaviors that are not immediately obvious from the UI but are important for complete test coverage.

#	Observation
1	**Activation code and deactivation code may be the same value.** The application does not prevent this. A user could set both to `1234`.
2	**Deactivation attempts are counted even when the bomb is Active but the timer has not expired.** A user could exhaust all attempts before the timer runs out.
3	**The attempt counter persists across re-activations within the same session.** If the user deactivates successfully (counter resets to 0) and then re-activates, the counter starts fresh. However, if the user accrues attempts *before* a successful deactivation, those are cleared by that success.
4	**The timer freezes (not resets) on successful deactivation.** If the user activates, lets 10 seconds pass, then deactivates, the timer stops at 20 seconds (assuming a 30-second countdown). Re-activating will resume from 20 seconds, not from 30.
5	**The countdown field accepts decimal input.** Entering `10.9` results in a 10-second countdown due to `parseInt` truncation.
6	**There is no upper bound on the countdown duration.** The only enforced constraint is `minimum: 1`. Very large values are accepted.
7	**The activation code is not validated for uniqueness against the deactivation code.** Setting both to the same value means the code both activates and deactivates — the behavior depends on the current bomb state at the time of submission.
8	**Direct keyboard entry and keypad entry share the same code display but have slightly different behaviors.** Keypad appends one digit at a time and respects the 4-digit cap. Direct input replaces the entire `codeStack` on each change event.
9	**The error message area has a minimum height.** An empty error state does not cause layout shift; the space is always reserved.
10	**Detonation is checked by a polling interval (every 100ms), not by an exact event.** This means the displayed time at detonation may show `00:00` briefly before the screen transitions.